

Harava Web Client — Build Guide

Everything the web app needs to talk to the backend we've built: tenancy, auth, the DPoP + encryption security layers, and every service endpoint that's live today.

Written for the web (React / RN-web) client · Backend as of 12 Jul 2026 · Keep this handy while wiring up the API layer.

What's in here

1. The basics — base URL, tenancy, response shape
2. Auth flow — login, MFA, tokens, refresh
3. Getting in — invitations & onboarding
4. Security layers — payload encryption & DPoP
5. The services — every endpoint that's live
6. Error handling & status codes
7. Build order — a suggested checklist

1 · The basics

Base URL & how tenancy works

Every company (tenant) lives at its own subdomain. On the web that's the easy path — the app is served from `https://<slug>.harava-sandbox.com` (staging) and the backend figures out the tenant from the host. You don't send a tenant header on web at all.

The one exception is a handful of *pre-login* calls where you might not be on the tenant subdomain yet (e.g. a generic "enter your workspace" screen). If you ever call those from a non-tenant host, send `X-Tenant-ID: <tenantUuid>`. You get that UUID from the tenant lookup below. Once you're logged in, forget about it — the tenant is baked into the access token.

Web reality: if you serve the SPA per-subdomain, you basically never touch tenant headers. Just point `fetch` at the current origin and go.

The response envelope — read this once, internalize it

Every single response — success or failure — comes back in the same wrapper:

```
{
  "success": true,
  "data": { ... },           // your actual payload lives here
  "error": null,             // or { code, message, fieldErrors? } on failure
  "timestamp": "2026-07-12T10:20:30Z",
  "path": "/api/v1/..."
}
```

So the golden rule: on success read `response.data`; on failure read `response.error`. Every example below shows the *contents of* `data`, not the wrapper. Write one small fetch helper that unwraps this and you'll never think about it again:

```
async function api(path, opts = {}) {
  const res = await fetch(`${BASE}${path}`, {
    ...opts,
    headers: { "Content-Type": "application/json", ...(opts.headers || {}) },
  });
  const body = await res.json();
  if (!res.ok || !body.success) {
    throw Object.assign(new Error(body.error?.message || res.statusText), {
      status: res.status, code: body.error?.code, fieldErrors: body.error?.fieldErrors,
    });
  }
  return body.data;
}
```

2 · Auth flow

There is **no public signup**. People get in by invitation (next section). Once they have an account, here's the login dance:

Step by step

1. **POST** `/api/v1/auth/login` with `{ email, password }`. You get one of two things back.
2. If `mfaRequired: false` → you get `tokens` directly. Done.
3. If `mfaRequired: true` → you get `{ mfaMethod, mfaToken }`. Show the code screen, then **POST** `/api/v1/auth/mfa/verify` with `{ mfaToken, code }` → tokens.
4. Put `Authorization: Bearer <accessToken>` on every authenticated call.
5. On a `401` with an expired token, **POST** `/api/v1/auth/refresh` with `{ refreshToken }` → a *new* access + refresh pair. Refresh tokens rotate, so store the new one and throw the old away.
6. Logout: **POST** `/api/v1/auth/logout` with `{ refreshToken }` to revoke it server-side.

login response shape

```
// mfaRequired === false
{ "mfaRequired": false, "mfaMethod": null, "mfaToken": null,
  "tokens": { "accessToken": "...", "refreshToken": "...",
    "tokenType": "Bearer", "expiresIn": 900 } }

// mfaRequired === true
{ "mfaRequired": true, "mfaMethod": "TOTP", "mfaToken": "...", "tokens": null }
```

Where to keep tokens (web edition)

Access tokens are short-lived (15 min / `expiresIn: 900`). Refresh tokens are long-lived and sensitive — treat them like a password.

Don't dump the refresh token in `localStorage`. Anything in `localStorage` is readable by any XSS. Prefer an in-memory access token + a refresh token in a `Secure; HttpOnly; SameSite` cookie if we can set one server-side, or at minimum keep the refresh token out of JS-readable storage. If we're stuck with storage for now, note it as tech debt and lock down XSS hard (CSP, escape everything).

MFA setup (for the account settings screen)

These are all under `/api/v1/account/mfa` and require a logged-in user:

Call	Body	Returns
<code>POST /totp/setup</code>	—	<code>{ secret, otpauthUri, qrCodeDataUri }</code> — render the <code>qrCodeDataUri</code> as an <code></code>
<code>POST /totp/enable</code>	<code>{ code }</code>	activation result — confirms the 6-digit code from their authenticator
<code>POST /email/enable</code>	—	turns on email-based MFA
<code>POST /sms/enable</code>	—	turns on SMS-based MFA
<code>POST /otp/send</code>	—	sends an email OTP (for the email MFA challenge)
<code>POST /sms/send</code>	—	sends an SMS OTP

Password & email

Call	Body
<code>POST /api/v1/auth/password/forgot</code>	<code>{ email }</code> — always 200, sends a reset link
<code>POST /api/v1/auth/password/reset</code>	<code>{ token, newPassword }</code>
<code>POST /api/v1/account/password/change</code>	<code>{ currentPassword, newPassword }</code> (logged in)
<code>POST /api/v1/auth/verify-email</code>	<code>{ token }</code>
<code>POST /api/v1/auth/resend-verification</code>	<code>{ email }</code>

Password rule the backend enforces: 8–100 chars, at least one lowercase, one uppercase, one digit. Mirror that in your form validation so users don't bounce off a 400.

3 · Getting in — invitations & onboarding

New members arrive via an email link like

`https://<slug>.harava-sandbox.com/accept-invite?token=...`. On web that's just a route — no deep-link plumbing needed like on mobile. Build the `/accept-invite` page:

1. `GET /api/v1/auth/invitations/preview?token=...` → show the org name, role, and the email being invited so they know what they're accepting.
2. `POST /api/v1/auth/invitations/accept` with `{ token, password, firstName, lastName }` → returns access + refresh tokens. They're now signed in — store tokens and drop them into the app.

Admin side — inviting people (OWNER / ADMIN)

Under `/api/v1/account/invitations`:

Call	Body / notes
POST /	<code>{ email, role }</code> where role is <code>ADMIN</code> or <code>MEMBER</code>
GET /	list pending invitations
POST /{id}/resend	resend the email
DEL /{id}	revoke a pending invite

Invitation object: `{ id, email, role, status, expiresAt, createdAt }`. Owners themselves are provisioned by the platform admin, not through this endpoint.

4 · Security layers

Two optional layers sit on top of plain HTTPS. Both are server-toggled and both should be **OFF while you build the happy path**, then flipped on to validate the crypto. Build your client so each is a switch you can turn on, not a rewrite.

4a · Payload encryption (optional, toggleable)

When on, request/response *bodies* are AES-256-GCM encrypted on top of TLS. Controlled by `harava.payload-encryption.enabled` server-side.

The contract

- Send an encrypted request body with header `X-Payload-Encrypted: true`.
- To get an encrypted response on a call with no body (a GET), send `X-Accept-Encrypted: true`. Responses to an encrypted request are always encrypted back.
- The wire format is base64 of `IV(12 bytes) || ciphertext || GCM tag(16 bytes)`. An encrypted response has `Content-Type: application/octet-stream`; decrypted it's just JSON.

Testing uses a shared 32-byte key handed to clients out-of-band. The planned production upgrade is a per-session ECDH handshake so each client derives its own key — but the wire envelope stays identical, so your encrypt/decrypt code won't change.

Web Crypto — the whole thing

```
const KEY = /* CryptoKey imported from the shared 32-byte secret */;

async function encrypt(bodyObj) {
  const iv = crypto.getRandomValues(new Uint8Array(12));
  const pt = new TextEncoder().encode(JSON.stringify(bodyObj));
  const ct = new Uint8Array(await crypto.subtle.encrypt({ name: "AES-GCM", iv }, KEY, pt));
  const env = new Uint8Array(iv.length + ct.length);
  env.set(iv, 0); env.set(ct, iv.length); // IV || ciphertext+tag
  return btoa(String.fromCharCode(...env)); // base64 envelope
}

async function decrypt(base64) {
  const env = Uint8Array.from(atob(base64), c => c.charCodeAt(0));
  const iv = env.slice(0, 12), ct = env.slice(12);
  const pt = await crypto.subtle.decrypt({ name: "AES-GCM", iv }, KEY, ct);
  return JSON.parse(new TextDecoder().decode(pt));
}

const res = await fetch(`${BASE}/api/v1/auth/login`, {
  method: "POST",
  headers: { "Content-Type": "application/json", "X-Payload-Encrypted": "true" },
  body: await encrypt({ email, password }),
});
const data = res.headers.get("X-Payload-Encrypted") === "true"
  ? await decrypt(await res.text()) : await res.json();
```

Web Crypto's AES-GCM already appends the 16-byte tag to the ciphertext, which is exactly what the Java server expects — no tag-splitting needed.

4b · DPoP — sender-constrained tokens (the important one)

This is the piece worth understanding well, because it's what makes a stolen access token useless. Controlled by `harava.dpop.enabled`. Keep it off during early integration, then turn it on.

The idea in one paragraph

A browser SPA can't hide a shared secret — anything you ship to the browser is readable. So instead of a secret, **the client generates its own ECDSA key pair** and only the *public* key ever leaves the device. Every authenticated request carries a fresh, short-lived JWS "proof" signed by the private key. The server binds your tokens to your key's thumbprint, so a token lifted from the wire or from storage is worthless to anyone who doesn't hold your private key — and there was never a secret to leak in the first place.

Heads up on rollout: flipping DPoP on invalidates any tokens that were issued while it was off (they carry no key binding). The server answers those with `INVALID_DPOP_PROOF` and the client just logs in again. Plan for a one-time re-login when we flip the switch.

Step 1 — generate a key pair once, keep the private key non-extractable

Use **ES256 (ECDSA P-256)**. The `false` below is the whole point — it makes the private key non-extractable, so JS (and therefore any XSS or devtools) can never read it out. Persist the `CryptoKey` in IndexedDB.

```
// once, at first run (per browser):
const kp = await crypto.subtle.generateKey(
  { name: "ECDSA", namedCurve: "P-256" }, /* extractable */ false, ["sign"]);
// store kp.privateKey + kp.publicKey (CryptoKey objects) in IndexedDB.
// never export privateKey.
```

Step 2 — the proof generator

You attach a **DPoP** header to (a) the token endpoints — login, mfa/verify, refresh, accept-invite — and (b) every authenticated request. On authenticated requests you additionally include an **ath** claim binding the proof to the specific access token.

```
const b64url = buf => btoa(String.fromCharCode(...new Uint8Array(buf)))
  .replace(/\+/g, "-").replace(/\//g, "_").replace(/=+$/, "");

async function dpopProof(method, url, accessToken /* optional */) {
  const jwkFull = await crypto.subtle.exportKey("jwk", kp.publicKey);
  const jwk = { crv: "P-256", kty: "EC", x: jwkFull.x, y: jwkFull.y }; // public only
  const header = { typ: "dpop+jwt", alg: "ES256", jwk };
  const payload = { htm: method.toUpperCase(), htu: url,
    iat: Math.floor(Date.now() / 1000), jti: crypto.randomUUID() };
  if (accessToken) { // bind proof -> token
    const ath = await crypto.subtle.digest("SHA-256",
      new TextEncoder().encode(accessToken));
    payload.ath = b64url(ath);
  }
  const enc = o => b64url(new TextEncoder().encode(JSON.stringify(o)));
  const signingInput = `${enc(header)}${enc(payload)}`;
  const sig = await crypto.subtle.sign({ name: "ECDSA", hash: "SHA-256" },
    kp.privateKey, new TextEncoder().encode(signingInput));
  return `${signingInput}.${b64url(sig)}`; // WebCrypto gives raw R||S = exactly ES256
}

// authenticated GET
const url = `${BASE}/api/v1/tenant/companies`;
await fetch(url, {
  headers: { "Authorization": `Bearer ${accessToken}`,
    "DPoP": await dpopProof("GET", url, accessToken) },
});
```

WebCrypto's ECDSA sign already produces the raw **R||S** signature JWS ES256 wants, so there's no DER conversion. Fold **dpopProof()** into the same fetch helper from section 1 — one place adds **Authorization** + **DPoP**, and everything downstream just works.

What the proof looks like on the wire

```
header: { "typ": "dpop+jwt", "alg": "ES256", "jwk": { ...your PUBLIC key... } }
payload: { "htm": "GET",
  "htu": "https://.harava-sandbox.com/api/v1/tenant/companies",
  "iat": 1720700000, "jti": "",
  "ath": "" } // authed calls only
```

iat is in seconds. **jti** must be unique for every single proof — the server tracks used ones and rejects repeats (replay protection). The server checks: the signature against the embedded key, that the key's thumbprint matches the token binding, that **htm** / **htu** match this exact request, that **iat** is fresh (within ~30s), that **jti** is unused, and that **ath** matches the token.

DPoP-specific failures

Code	Status	What it means / what to do
INVALID_DPOP_PROOF	401	Missing / malformed / expired proof, wrong method-or-URI, or a proof not bound to this token. Also what you get for pre-DPoP tokens after the flag flips — just re-login.
REPLAYED_REQUEST	401	That <code>jti</code> was already used. Never reuse a proof — generate a fresh one per request.
SERVICE_UNAVAILABLE	503	Server couldn't reach its replay store. Safe to retry with a new proof.

Note: right now access tokens are DPoP-bound. Refresh-token binding is a documented follow-up on the backend, so nothing extra for you to do on refresh beyond sending a proof.

5 · The services that are live

All of these are under `/api/v1`, need `Authorization: Bearer` (plus a DPoP proof once that's on), and are role-gated. Roles in the system: `OWNER`, `ADMIN`, `MEMBER`, `CLIENT` (external portal user), and `PLATFORM_ADMIN` (separate console).

Companies — OWNER / ADMIN · `/api/v1/tenant/companies`

Endpoint	Body	Returns
POST /	<code>{ name, externalRef? }</code>	CompanyResponse
GET /	—	CompanyResponse[]
GET /{id}	—	CompanyResponse
PATCH /{id}	<code>{ name?, externalRef? }</code>	CompanyResponse
POST /{id}/suspend	—	CompanyResponse (status flips)
POST /{id}/activate	—	CompanyResponse
GET /{id}/users	—	CompanyUserResponse[]
POST /{id}/users/invite	<code>{ email }</code>	InvitationResponse

```
// CompanyResponse
{ "id": "uuid", "name": "Acme Ltd", "externalRef": "QB-1029",
  "status": "ACTIVE", "quickbooksConnected": true,
  "userCount": 4, "createdAt": "2026-07-01T09:00:00Z" }

// CompanyUserResponse
{ "id": "uuid", "email": "a@acme.com", "fullName": "Ada Lovelace",
  "enabled": true, "lastLoginAt": "2026-07-11T14:00:00Z" }
```

Insights & reports — OWNER / ADMIN · /api/v1/tenant/companies/{companyId}

Endpoint	Returns
GET /insights/summary	headline numbers for the dashboard
GET /insights/cashflow	time series of inflow / outflow / net
GET /insights/pnl	time series of revenue / costs / net
GET /reports	list of available reports

```
// InsightSummaryResponse
{ "companyId": "uuid", "companyName": "Acme Ltd", "currency": "NGN",
  "asOf": "2026-07-12", "cashPosition": 1250000.00,
  "incomeThisMonth": 480000.00, "expensesThisMonth": 310000.00,
  "netProfitThisMonth": 170000.00, "source": "QUICKBOOKS" }

// CashflowResponse (PnlResponse is identical shape w/ revenue/costs/net)
{ "companyId": "uuid", "currency": "NGN", "source": "QUICKBOOKS",
  "points": [ { "period": "2026-06", "inflow": 500000.00,
    "outflow": 300000.00, "net": 200000.00 }, ... ] }

// ReportSummaryResponse[]
[ { "key": "pnl", "title": "Profit & Loss", "period": "2026-Q2",
  "available": true }, ... ]
```

Charting tip: `cashflow.points` and `pnl.points` are already period-bucketed and ordered — feed `points` straight into your chart lib. `source` tells you if the numbers are real (QuickBooks) or placeholder, so you can badge the dashboard accordingly.

QuickBooks connection — OWNER / ADMIN · /api/v1/tenant/companies/{companyId}/quickbooks

Endpoint	Returns
GET /	ConnectionResponse — current status
POST /connect	ConnectionResponse — kicks off / records the connection
DEL /	ConnectionResponse — disconnect

```
// ConnectionResponse
{ "companyId": "uuid", "status": "CONNECTED", "realmId": "4620816...",
  "connectedAt": "2026-07-01T09:00:00Z", "lastSyncedAt": "2026-07-12T06:00:00Z" }
```

Client portal — CLIENT role · /api/v1/portal

This is the external-client view — a company's own client logs in and sees just their numbers. Same insight shapes as above, but no `companyId` in the path (it's inferred from who they are).

Endpoint	Returns
GET /me	PortalProfileResponse
GET /insights/summary	InsightSummaryResponse
GET /insights/cashflow	CashflowResponse
GET /insights/pnl	PnlResponse
GET /reports	ReportSummaryResponse[]

```
// PortalProfileResponse
{ "userId": "uuid", "email": "client@co.com", "fullName": "Grace Hopper",
  "companyId": "uuid", "companyName": "Acme Ltd", "firmName": "Harava Books",
  "currency": "NGN" }
```

Practically: build one insights component and reuse it for both the OWNER/ADMIN company view and the CLIENT portal — only the base path and role differ.

Account & audit

Endpoint	Role	Returns
GET /api/v1/account/me	any	UserResponse — the logged-in user's profile
GET /api/v1/account/audit	OWNER / ADMIN	paginated audit events

```
// UserResponse
{ "id": "uuid", "tenantId": "uuid", "email": "a@acme.com",
  "firstName": "Ada", "lastName": "Lovelace", "phoneNumber": "+234...",
  "emailVerified": true, "mfaEnabled": true, "mfaMethod": "TOTP" }

// GET /account/audit?page=0&size=20 -> Spring Page
{ "content": [ { "id": "uuid", "eventType": "LOGIN", "outcome": "SUCCESS",
  "actorUserId": "uuid", "actorEmail": "a@acme.com", "ipAddress": "102.89...",
  "description": "User logged in", "occurredAt": "2026-07-12T10:00:00Z" } ],
  "totalElements": 143, "totalPages": 8, "number": 0, "size": 20 }
```

Audit is a standard Spring **Page** — use **page** & **size** query params, and read **totalPages** / **number** for your pager.

Also on the backend but probably not your first sprint: the platform-admin console (**/api/v1/platform/***, role **PLATFORM_ADMIN**) for creating tenants and provisioning owners — separate login at **/api/v1/platform/auth/login**. Worth a separate mini-app rather than bolting it onto the tenant SPA.

6 · Error handling

Failures come back in the same envelope with **success: false** and an **error** object: **{ code, message, fieldErrors? }**. **fieldErrors** shows up on validation failures — a map of field → message you can drop straight onto form inputs. Switch on **error.code**, not the message text.

code	HTTP	When you'll hit it
<code>VALIDATION_ERROR</code>	400	Bad/missing fields — check <code>fieldErrors</code>
<code>BAD_REQUEST</code>	400	Malformed request
<code>UNAUTHORIZED</code> / <code>INVALID_CREDENTIALS</code>	401	Not logged in / wrong email-password
<code>MFA_REQUIRED</code>	401	Finish the MFA step
<code>INVALID_MFA_CODE</code>	401	Wrong / expired MFA code
<code>TOKEN_EXPIRED</code>	401	Access token aged out → refresh
<code>TOKEN_INVALID</code>	401	Garbage / revoked token → re-login
<code>INVALID_DPOP_PROOF</code> / <code>REPLAYED_REQUEST</code>	401	DPoP issues — see section 4b
<code>ACCOUNT_LOCKED</code>	423	Too many bad logins
<code>ACCOUNT_DISABLED</code> / <code>EMAIL_NOT_VERIFIED</code> / <code>FORBIDDEN</code>	403	Not allowed / needs verification
<code>NOT_FOUND</code>	404	No such resource
<code>CONFLICT</code> / <code>EMAIL_ALREADY_REGISTERED</code> / <code>MFA_ALREADY_ENABLED</code> / <code>MFA_NOT_ENABLED</code>	409	State conflict
<code>TOO_MANY_REQUESTS</code>	429	Rate limited — back off & retry
<code>INTERNAL_ERROR</code>	500	Our bug — surface a generic message
<code>SERVICE_UNAVAILABLE</code>	503	Transient — retry

One interceptor to rule them all: in your fetch wrapper, catch `401 + TOKEN_EXPIRED` → auto-refresh once and replay the original request; catch `429` → back off; anything with `fieldErrors` → hand back to the form. Do it once, centrally.

7 · Suggested build order

1. **Fetch wrapper first.** Unwrap the envelope, throw typed errors. Everything sits on this.
2. **Auth happy path** with both flags OFF: login → store tokens → hit `GET /account/me` → render something gated. Add the refresh-on-401 loop.
3. **MFA + invitation accept + password reset** pages.
4. **The real screens:** companies list/detail, insights dashboard (summary + cashflow + pnl charts), QuickBooks connect card, audit log, client portal.
5. **Turn on payload encryption** in staging, validate the encrypt/decrypt path, leave it toggleable.
6. **Turn on DPoP** in staging: key-gen on first run, proof on token calls + every authed request, handle the one-time re-login. This is the last layer because it's easiest to debug once everything else is solid.

Golden rule for both security layers: build them as switches, not rewrites. If your fetch wrapper is the single place that adds headers and does encrypt/decrypt, flipping either flag is a two-line change — not a refactor.

Questions or something not matching the backend? The source of truth is the Postman collection

(`Harava.postman_collection.json`) and the `DPOP_DESIGN.md` / `CLIENT_INTEGRATION.md` notes in the repo.

This guide is the friendly version of those.